

Docker & Docker Compose:

Comprehensive Notes

1. Introduction to Docker

What is Docker? Docker is a virtualization software that packages applications and their dependencies into standardized units called **containers**. These containers hold everything the application needs to run: the code, runtime, libraries, environment variables, and configurations.

Problems it solves:

- **Development:** Before containers, developers had to install required services (like PostgreSQL, Redis) directly on their OS, leading to version conflicts and tedious setup processes. Docker allows developers to pull services as isolated containers without installing binaries locally.
- **Deployment:** Historically, operations teams manually configured servers based on developer instructions, often leading to "it works on my machine" issues. Docker standardizes this process by handing operations a containerized package that runs uniformly on any server.

Docker vs. Virtual Machines (VMs)

While both are virtualization tools, they operate at different levels of the operating system.

- **Virtual Machines:** Virtualize the *entire hardware*. Every VM boots its own guest operating system kernel on top of a hypervisor. They are heavy, take gigabytes of space, and require minutes to boot.
- **Docker:** Virtualizes the *application layer*. Containers share the host operating system's kernel. They are lightweight (often a few megabytes), fast (start in milliseconds), and consume fewer resources.

Note on Compatibility: Because Docker containers share the host's kernel, running a Linux container natively requires a Linux kernel. Tools like *Docker*

Desktop run a lightweight Linux VM (hypervisor layer) on Windows/macOS to provide the necessary Linux kernel.

2. Core Concepts

- **Image:** The blueprint or application artifact. An image contains the compiled application code, environment configuration, operating system application layer, and required runtime tools (e.g., Node.js, Java). It is static and portable.
- **Container:** A running instance of an image. When you execute an image on an OS, it spins up an isolated environment called a container. You can run multiple containers from a single image.
- **Docker Registry:** A storage system for Docker images.
 - **Docker Hub:** The largest public registry for official and community-maintained images.
 - **Private Registries:** Used by companies to store proprietary images (e.g., AWS ECR, Nexus).
- **Tags:** Versions attached to an image (e.g., `nginx:1.23`). If no tag is specified, Docker defaults to the `latest` tag.

3. Creating Custom Images (Dockerfile)

To containerize your own application, you write a **Dockerfile**. It provides step-by-step instructions for building a Docker image.

```
FROM node:19-alpine          # Base image
ENV PORT=3000                # Set environment variable
WORKDIR /app                 # Set the working directory in the container
COPY package.json .          # Copy specific files from host to container
RUN npm install               # Execute Linux/terminal commands during build
COPY . .                     # Copy all source files
CMD ["node", "server.js"]    # Entry point command when the container starts
```

To build an image from a Dockerfile: `docker build -t my-app:1.0 .`

4. Data Persistence (Docker Volumes)

By default, containers are ephemeral. If a container is removed, all changes within its virtual file system are lost. **Docker Volumes** solve this by mapping a physical folder on the host machine to a directory inside the container's virtual file system.

- **Host Volumes:** You specify the exact path on your host machine to bind to the container (`-v /path/on/host:/path/in/container`).
- **Anonymous Volumes:** Docker auto-generates a storage directory managed completely by Docker.
- **Named Volumes:** You specify a logical name (e.g., `mongo-data`), and Docker manages the actual physical path on the host. This is the recommended approach for production databases.

5. Essential Docker Commands

Command	Description
<code>docker pull <image></code>	Downloads an image from a registry to the local machine.
<code>docker run <image></code>	Creates and starts a new container from an image.
<code>docker ps</code>	Lists running containers. Add <code>-a</code> to see stopped containers as well.
<code>docker stop <id></code>	Gracefully stops a running container.
<code>docker start <id></code>	Restarts a stopped container, retaining its initial configurations.
<code>docker rm <id></code>	Removes a stopped container.
<code>docker rmi <image></code>	Removes a local image.
<code>docker logs <id></code>	Outputs the application logs generated inside the container.
<code>docker exec -it <id></code> <code>sh</code>	Opens an interactive terminal session inside a running container.

Important `docker run` Flags

- `-d` : Detached mode. Runs the container in the background.
- `-p <host_port>:<container_port>` : Port binding. Exposes the container's internal port to a specific port on the localhost (e.g., `-p 9000:80`).
- `--name <name>` : Assigns a custom name to the container instead of a random hash.
- `-e <KEY>=<VALUE>` : Passes environment variables to the container.
- `--net <network_name>` : Attaches the container to a specific isolated Docker network.

6. Docker Compose

What is it? When applications consist of multiple interconnected services (e.g., a Node.js backend, a MongoDB database, and a Redis cache), deploying and managing them via individual `docker run` commands becomes tedious and error-prone. Docker Compose is a tool that allows you to define, configure, and run multi-container Docker applications using a single YAML file.

Benefits of Docker Compose:

- Configuration is stored as code (`docker-compose.yml`).
- Automatically creates a shared Docker Network for the defined services, allowing them to communicate via service names instead of IP addresses.
- Makes starting, stopping, and cleaning up an entire environment achievable with single commands.

Example `docker-compose.yml`

```
version: '3.8'
services:
  mongodb:
    image: mongo:latest
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_ROOT_USERNAME=${MONGO_USER}
      - MONGO_INITDB_ROOT_PASSWORD=${MONGO_PASS}
    volumes:
      - mongo-data:/data/db

  mongo-express:
    image: mongo-express
```

```
ports:
  - "8080:8081"
environment:
  - ME_CONFIG_MONGODB_ADMINUSERNAME=${MONGO_USER}
  - ME_CONFIG_MONGODB_ADMINPASSWORD=${MONGO_PASS}
  - ME_CONFIG_MONGODB_SERVER=mongodb
depends_on:
  - mongodb

my-app:
  build: .
  ports:
    - "3000:3000"
  depends_on:
    - mongodb

volumes:
  mongo-data:
    driver: local
```

Key Features Shown:

- **Dynamic Variables:** Instead of hardcoding secrets, use `${VAR_NAME}` to pull from the host's environment variables.
- **depends_on:** Dictates start order. `mongo-express` will wait for `mongodb` to start first.
- **Custom Build:** `my-app` builds its image locally using the Dockerfile in the current directory (`build: .`) instead of pulling a pre-built image.

Docker Compose Commands

- `docker-compose up -d` : Creates and starts all services, networks, and volumes in the background.
- `docker-compose down` : Stops and completely removes all services and networks defined in the compose file. (Volumes persist).
- `docker-compose stop` : Stops the services without removing them. Data in non-persistent directories is preserved.
- `docker-compose start` : Restarts services that were previously stopped.

7. The Big Picture Workflow

How does Docker fit into a modern CI/CD Pipeline?

1. **Local Development:** A developer creates code and a `Dockerfile`. They might use `docker-compose` to quickly spin up dependencies like databases locally.
2. **Continuous Integration (CI):** The developer pushes code to a Git repository. A CI server (like Jenkins or GitHub Actions) automatically triggers, compiles the app, and builds a Docker image using the `Dockerfile`.
3. **Registry Push:** The CI server authenticates and pushes the newly built image (tagged with a version) to a Docker Registry (e.g., AWS ECR).
4. **Deployment:** The target environment (QA, Staging, Production) pulls the image from the registry and runs the container. Often, `docker-compose` or an orchestration tool like **Kubernetes** manages the deployment of these containers across clusters of servers.